

DATA ANALYST | SQL INTERVIEW | EXAM PREP

SQL Database Fundamentals

DDL • Triggers • Relationships • OLTP & OLAP

Data Analyst Interviews

SQL Interviews

Microsoft / Fabric Certifications

College Exams

Practical SQL Projects

Table of Contents

- 1 DDL — Data Definition Language
- 2 SQL Relationships
- 3 Triggers
- 4 OLTP — Online Transaction Processing
- 5 OLAP — Online Analytical Processing
- 6 OLTP vs OLAP
- 7 Complete Data Analytics Architecture
- 8 Normalization & Denormalization
- 9 Fact and Dimension Tables
- 10 Interview & Exam Questions (30) + Answer Key
- 11 Quick Revision Sheet
- 12 Final Revision Checklist

HOW TO USE THIS HANDBOOK

Read each section top to bottom the first time. On revision day, skip straight to the **Interview Trap boxes**, the **comparison tables**, and Section 11 (Quick Revision Sheet). Every highlighted term (**PRIMARY KEY**, **TRIGGER**, **OLTP**, **OLAP** ...) is something interviewers commonly ask you to define in one line — practice saying the definition out loud.

Section 1 — DDL (Data Definition Language)

CREATE • Data Types • Constraints • ALTER • DROP vs TRUNCATE vs DELETE

1. What is DDL?

Simple explanation: DDL is the part of SQL you use to **build and shape** the database itself — creating databases, tables, and views, and defining what a table is allowed to store. Think of DDL as "designing the shelves," while DML is "putting items on the shelves."

Technical definition: **DDL (Data Definition Language)** is the subset of SQL statements used to define, modify, and remove the structure of database objects such as databases, tables, views, indexes, and schemas.

Purpose

- Define the structure/schema of the database (tables, columns, data types).
- Enforce structural rules (constraints) before any data is ever inserted.
- Create logical objects like views for reporting.

When is DDL used?

- At the start of a project, when designing the schema.
- When the business requirement changes and a table needs a new column.
- When cleaning up — dropping unused tables/views.

DDL vs DML vs DQL vs DCL vs TCL

Category	Full Form	Commands	What it does
DDL	Data Definition Language	CREATE, ALTER, DROP, TRUNCATE, RENAME	Defines/changes structure
DML	Data Manipulation Language	INSERT, UPDATE, DELETE	Changes data inside tables
DQL	Data Query Language	SELECT	Reads/retrieves data
DCL	Data Control Language	GRANT, REVOKE	Controls access/permissions
TCL	Transaction Control Language	COMMIT, ROLLBACK, SAVEPOINT	Manages transactions

INTERVIEW TRAP

DDL statements are **auto-committed** in most databases (MySQL, Oracle) — you generally cannot roll back a CREATE/DROP/ALTER/TRUNCATE the way you can roll back an INSERT/UPDATE/DELETE. This single fact is asked in almost every SQL interview.

2. CREATE

CREATE DATABASE

```
CREATE DATABASE company_db;
USE company_db;
```

CREATE TABLE

```
CREATE TABLE employees (
  emp_id      INT PRIMARY KEY,
  first_name  VARCHAR(50) NOT NULL,
  last_name   VARCHAR(50) NOT NULL,
  salary      DECIMAL(10,2) DEFAULT 0,
  dept_id     INT,
  hire_date   DATE
);
```

CREATE VIEW

A **VIEW** is a saved, virtual table — a stored SELECT query you can reuse like a table. It doesn't store data itself (in a normal view); it just runs the underlying query every time you use it.

```
CREATE VIEW high_earners AS
SELECT emp_id, first_name, last_name, salary
FROM employees
WHERE salary > 80000;

-- Usage
SELECT * FROM high_earners;
```

WHY USE VIEWS?

1) Hide complexity of long joins from end users. 2) Provide a restricted "window" of data (e.g. hide salary column from HR reports). 3) Reuse common query logic across reports/BI tools like Power BI.

3. SQL Data Types

Data Type	Stores	When to use it
INT	Whole numbers	IDs, counts, quantities — anything without decimals.
DECIMAL(p, s)	Exact fixed-point numbers	Money/salary/prices — where rounding errors are unacceptable.
FLOAT	Approximate floating-point numbers	Scientific/measurement data where tiny rounding is acceptable. Avoid for currency.
VARCHAR(n)	Variable-length text, up to n chars	Names, emails, addresses — text that varies in length.
CHAR(n)	Fixed-length text, always n chars (padded)	Fixed-size codes: country code "IN", gender "M/F", status flags.
DATE	Calendar date (YYYY-MM-DD)	Birthdates, hire dates, order dates — date only, no time.
DATETIME	Date + time	Timestamps: order placed at, login time, audit logs.
BOOLEAN	TRUE/FALSE (stored as TINYINT(1) in MySQL)	Flags: is_active, is_deleted, is_verified.

INTERVIEW TRAP

DECIMAL vs FLOAT: Never use FLOAT for money. FLOAT is a binary approximation and can produce rounding errors (e.g., $0.1 + 0.2 \neq 0.3$ exactly). DECIMAL stores exact digits, so it is always correct for currency.

4. Constraints

Constraints are rules attached to a column (or table) that the database enforces automatically, so bad data never even makes it into the table.

Constraint	Meaning	Example
PRIMARY KEY	Uniquely identifies each row. Cannot be NULL, must be unique.	<code>emp_id INT PRIMARY KEY</code>
FOREIGN KEY	Links a column to a primary key in another table; enforces referential integrity.	<code>dept_id INT REFERENCES departments(dept_id)</code>
NOT NULL	Column must always have a value.	<code>first_name VARCHAR(50) NOT NULL</code>
UNIQUE	No two rows can have the same value (NULLs usually allowed).	<code>email VARCHAR(100) UNIQUE</code>
DEFAULT	Auto-fills a value if none is given.	<code>status VARCHAR(10) DEFAULT 'ACTIVE'</code>
CHECK	Validates values against a condition.	<code>salary DECIMAL(10,2) CHECK (salary > 0)</code>

Using multiple constraints together

```
CREATE TABLE employees (
  emp_id INT PRIMARY KEY,
  email VARCHAR(100) UNIQUE NOT NULL,
  salary DECIMAL(10,2) DEFAULT 0 CHECK (salary >= 0),
  dept_id INT,
  status VARCHAR(10) DEFAULT 'ACTIVE' NOT NULL,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

KEY POINT

A table can have only **one PRIMARY KEY**, but it can have **many UNIQUE** constraints and **many FOREIGN KEYS**.

5. ALTER

`ALTER TABLE` changes the structure of an already-existing table.

```
-- Add a column
ALTER TABLE employees ADD phone VARCHAR(15);

-- Modify a column's data type / size (MySQL)
ALTER TABLE employees MODIFY phone VARCHAR(20);

-- Alter a column's type (Standard SQL / SQL Server / PostgreSQL)
ALTER TABLE employees ALTER COLUMN phone VARCHAR(20); -- SQL Server style
ALTER TABLE employees ALTER COLUMN phone TYPE VARCHAR(20); -- PostgreSQL style

-- Drop a column
ALTER TABLE employees DROP COLUMN phone;

-- Rename a column (MySQL 8+)
ALTER TABLE employees RENAME COLUMN phone TO mobile_no;

-- Rename a table
ALTER TABLE employees RENAME TO staff;
```

SYNTAX DIFFERS BY DATABASE!

MySQL uses `MODIFY` to change a column's type. **SQL Server** and **PostgreSQL** use `ALTER COLUMN` instead (and PostgreSQL needs the extra keyword `TYPE`). Interviewers love to ask "what's wrong with this ALTER statement" — check which database is implied by the question.

6. DROP vs TRUNCATE vs DELETE

Feature	DROP	TRUNCATE	DELETE
Type of command	DDL	DDL	DML
Removes rows	Yes (indirectly, table is gone)	Yes, all rows	Yes, all or filtered rows
Removes table structure	Yes — table itself is deleted	No — structure stays, empty table remains	No — structure and rows (not matched) stay
WHERE allowed	No	No	Yes
Speed	Fast	Very fast (deallocates data pages)	Slower (logs each row)
Typical use	Remove a table permanently	Empty a table quickly, keep structure	Remove specific rows
Transaction behavior	Auto-committed, usually can't roll back	Auto-committed in most RDBMS, minimal logging	Can be rolled back (logged row-by-row)
Resets AUTO_INCREMENT	N/A (table gone)	Usually yes	No
Triggers fired	No	No (in most RDBMS)	Yes

COMMON INTERVIEW TRAPS

- "TRUNCATE is DML" — **False**. TRUNCATE is DDL, even though it only affects data, because it deallocates storage rather than logging individual row deletes.
- "DELETE always needs a WHERE clause" — False, `DELETE FROM table;` without WHERE deletes all rows (but keeps structure), and it CAN be rolled back inside a transaction (unlike TRUNCATE in many databases).
- "TRUNCATE fires DELETE triggers" — False in most databases (MySQL, SQL Server, PostgreSQL) — triggers generally do **not** fire on TRUNCATE.

Section 2 — SQL Relationships

1:1 • 1:N • M:N • Primary Key vs Foreign Key

1. One-to-One (1:1)

Each row in Table A relates to **exactly one** row in Table B, and vice versa.



Example: One person has exactly one passport, and one passport belongs to exactly one person.

```
CREATE TABLE person (  
    person_id INT PRIMARY KEY,  
    name      VARCHAR(50)  
);  
  
CREATE TABLE passport (  
    passport_id INT PRIMARY KEY,  
    passport_no VARCHAR(20) UNIQUE,  
    person_id   INT UNIQUE,      -- UNIQUE enforces the "1:1"  
    FOREIGN KEY (person_id) REFERENCES person(person_id)  
);
```

WHY IS THIS USEFUL?

1:1 is often used to split a very wide table (e.g. separating rarely-used or sensitive columns like passport/medical details into their own table) or to model a genuine real-world one-to-one fact.

2. One-to-Many (1:N)

One row in Table A can relate to **many** rows in Table B, but each row in B relates to only one row in A.



```
CREATE TABLE customer (  
    customer_id INT PRIMARY KEY,  
    name        VARCHAR(50)  
);  
  
CREATE TABLE orders (  
    order_id     INT PRIMARY KEY,  
    order_date   DATE,  
    customer_id  INT,          -- FK lives on the "many" side  
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)  
);
```

WHY DOES THE FOREIGN KEY GO ON THE "MANY" SIDE?

Each row on the "many" side (an order) can only point to ONE parent (one customer), so a single FK column is enough. If the FK were placed on the "one" side (customer), a customer with multiple orders would need multiple values in one column — which a normal column cannot hold.

3. Many-to-Many (M:N)

Many rows in Table A can relate to many rows in Table B. This needs a **junction (bridge) table** in the middle.



```

CREATE TABLE student (
    student_id INT PRIMARY KEY,
    name VARCHAR(50)
);

CREATE TABLE course (
    course_id INT PRIMARY KEY,
    title VARCHAR(50)
);

CREATE TABLE enrollment (
    -- junction / bridge table
    student_id INT,
    course_id INT,
    enroll_date DATE,
    PRIMARY KEY (student_id, course_id), -- composite primary key
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (course_id) REFERENCES course(course_id)
);

```

PURPOSE OF THE JUNCTION TABLE

A plain FK column can only store one value, so it cannot represent "this student takes 5 courses AND this course has 200 students" directly. The junction table breaks the M:N relationship into two 1:N relationships (STUDENT → ENROLLMENT and COURSE → ENROLLMENT), and can also carry extra data about the relationship itself (e.g. enroll_date, grade).

4. Primary Key vs Foreign Key

Aspect	Primary Key	Foreign Key
Definition	Uniquely identifies each row in its own table	References a primary key in another (or the same) table
Uniqueness	Must be unique	Can repeat (many rows can share the same FK value)
NULL allowed?	Never	Usually yes (unless also NOT NULL)
Per table	Only one PK (can be composite)	Can have multiple FKs
Purpose	Identity of a row	Relationship / referential integrity between tables

Referential integrity means the database will not allow a foreign key value to point to a row that doesn't exist. E.g. you cannot insert an order with `customer_id = 999` if no customer 999 exists.

CONFUSING INTERVIEW QUESTIONS

Q: Can a foreign key reference a non-primary-key column?

A: Yes — it must reference a column with a UNIQUE constraint (or PRIMARY KEY), not necessarily the PK itself.

Q: Can a table have a foreign key to itself?

A: Yes — a "self-referencing" FK, e.g. an `employees` table with `manager_id` referencing `emp_id` in the same table.

Q: Can a primary key be made of multiple columns?

A: Yes — a **composite primary key**, commonly used in junction tables.

Section 3 — Triggers

Events • Timing • OLD/NEW • Syntax • Use Cases • Disadvantages

1. What is a Trigger?

DEFINITION

A **TRIGGER** is a database object that automatically executes a block of SQL code when a specified event (INSERT, UPDATE, or DELETE) happens on a table — you never call it directly, the database calls it for you.

Why triggers are used: to automatically enforce rules or side-effects that must *always* happen, no matter which application or user changes the data — e.g. always logging a salary change, always keeping a summary total up to date, always validating a value before it's saved.

2. Trigger Events

INSERT → row(s) added
UPDATE → row(s) changed
DELETE → row(s) removed

3. Trigger Timing

BEFORE → runs BEFORE the change is applied to the table
AFTER → runs AFTER the change is applied to the table

Combination	Typical Use
BEFORE INSERT	Validate/clean data, auto-generate a value, before the row is saved.
AFTER INSERT	Log the new row, update a running total, send a notification flag.
BEFORE UPDATE	Validate new values, prevent an invalid update, auto-set an "updated_at" field.
AFTER UPDATE	Record the old vs new value into an audit table (most common use case).
BEFORE DELETE	Block deletion under certain conditions, archive the row first.
AFTER DELETE	Log what was deleted, update a summary/count table.

4. OLD and NEW

OLD → the row's value BEFORE the change
NEW → the row's value AFTER the change

Event	OLD available?	NEW available?
INSERT	No (row didn't exist before)	Yes
UPDATE	Yes	Yes
DELETE	Yes	No (row won't exist after)

5. CREATE TRIGGER — Full Worked Example

Scenario: Every time an employee's salary is updated, automatically record the old salary, the new salary, and when the change happened, into an audit table.

```
-- Step 1: main table
CREATE TABLE employees (
    emp_id    INT PRIMARY KEY,
    name      VARCHAR(50),
    salary    DECIMAL(10,2)
);

-- Step 2: audit table
CREATE TABLE employee_audit (
    audit_id   INT AUTO_INCREMENT PRIMARY KEY,
    emp_id     INT,
    old_salary  DECIMAL(10,2),
    new_salary  DECIMAL(10,2),
    changed_at DATETIME
);

-- Step 3: the trigger
DELIMITER $$

CREATE TRIGGER trg_salary_audit
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    IF OLD.salary <> NEW.salary THEN
        INSERT INTO employee_audit (emp_id, old_salary, new_salary, changed_at)
        VALUES (OLD.emp_id, OLD.salary, NEW.salary, NOW());
    END IF;
END$$

DELIMITER ;
```

Line-by-line explanation

- `DELIMITER $$` — temporarily changes the statement terminator from `;` to `$$` so the semicolons inside the trigger body don't end the `CREATE TRIGGER` statement early.
- `CREATE TRIGGER trg_salary_audit` — names the trigger.
- `AFTER UPDATE ON employees` — this trigger runs after any `UPDATE` on the `employees` table.
- `FOR EACH ROW` — the trigger body runs once per row that was updated (row-level trigger).
- `IF OLD.salary <> NEW.salary THEN` — only log if the salary actually changed (skip if some other column was updated).
- `INSERT INTO employee_audit ...` — writes a new audit row using `OLD.emp_id`, `OLD.salary` (before) and `NEW.salary` (after).
- `NOW()` — captures the exact timestamp of the change.
- `END$$` / `DELIMITER ;` — closes the trigger body and restores the normal delimiter.

6. Real-World Trigger Use Cases

- **Audit logging:** track who changed what and when (as above).
- **Maintaining history:** keep old versions of a row in a history table.
- **Automatic validation:** reject or adjust values that don't meet business rules before saving.
- **Maintaining summary tables:** update a running total (e.g. total stock quantity) whenever a detail row changes.
- **Tracking changes:** flag rows as "needs sync" for downstream systems.

7. Trigger Disadvantages

WHY TRIGGERS CAN BE DANGEROUS

- **Hidden logic:** a developer running a simple `UPDATE` may not realize extra logic is silently firing.
- **Debugging difficulty:** errors appear to come from a plain `INSERT/UPDATE`, not from the trigger itself.
- **Performance impact:** every row-level operation now does extra work, which can slow bulk loads.
- **Maintenance problems:** business logic gets scattered across the database instead of the application.
- **Excessive/chained triggers:** a trigger firing another trigger can cause unpredictable cascades.

8. Trigger Interview Traps

Q: Can a trigger be called manually?

A: No. Triggers cannot be invoked directly by the user — they fire automatically only in response to their defined event (`INSERT/UPDATE/DELETE`). This is the #1 difference from stored procedures.

Q: What is OLD? What is NEW?

A: `OLD` refers to the row's values before the change; `NEW` refers to the values after the change. `OLD` is unavailable on `INSERT`, `NEW` is unavailable on `DELETE`.

Q: BEFORE vs AFTER?

A: `BEFORE` triggers can modify the incoming `NEW` values before they're saved (good for validation/defaults). `AFTER` triggers see the final saved values (good for logging/auditing, since the change is already committed to the table).

Q: Trigger vs Stored Procedure?

A: A stored procedure is called explicitly by a user/application (`CALL my_proc()`). A trigger fires implicitly, automatically, in response to a data event — it can never be called directly.

Q: Trigger vs Constraint?

A: A constraint (`CHECK`, `FOREIGN KEY`, `NOT NULL`) is a simple, declarative rule enforced by the engine with minimal overhead. A trigger is procedural code that can implement much more complex logic, but is heavier and harder to trace. Prefer a constraint whenever the rule is simple enough to express as one.

Section 4 — OLTP (Online Transaction Processing)

Definition • Characteristics • Architecture

DEFINITION

OLTP systems manage large numbers of short, fast, everyday transactions — inserting, updating, and deleting individual records in real time to run the day-to-day operations of a business.

Purpose

Keep the business running *right now*: process a sale, update an account balance, confirm a booking.

Characteristics

- Large number of short, simple transactions (INSERT/UPDATE/DELETE)
- Requires fast response time (milliseconds)
- Data is current / up-to-date, not historical
- High concurrency — many users/transactions at the same time
- Data is highly **normalized** to avoid redundancy and keep writes fast/consistent
- Uses **ACID** transactions (Atomicity, Consistency, Isolation, Durability)

Typical Operations

```
INSERT new order
UPDATE account balance
DELETE cancelled booking
SELECT single customer record
```

Examples

Bank transaction	→ debit/credit an account instantly
Amazon order	→ placing a single order
ATM withdrawal	→ updating balance in real time
Flight booking	→ reserving one seat
Employee attendance	→ punching in/out

Simple OLTP Architecture

```
graph TD
    UserApp["[User / App]"] --> OLTPDB["[OLTP Database] — normalized tables (3NF)"]
    OLTPDB --> FastOps["Fast INSERT / UPDATE / DELETE  
(order placed, payment processed, seat booked)"]
```

Section 5 — OLAP (Online Analytical Processing)

Definition • Characteristics • Architecture

DEFINITION

OLAP systems are built for analyzing large volumes of historical data — running complex queries with aggregations, trends, and comparisons to support business decisions.

Purpose

Understand what *has happened* over time, to guide decisions: which region sold more, which product is trending, how revenue grew year over year.

Characteristics

- Small number of complex, long-running queries
- Reads large volumes of **historical** data (months/years)
- Heavy use of **aggregations**: SUM, AVG, COUNT, GROUP BY
- Data is often **denormalized** (star/snowflake schema) for faster reads
- Data is loaded periodically (batch ETL/ELT), not written transaction-by-transaction
- Lives in a **data warehouse**

Examples

Monthly revenue	→ SUM(sales) grouped by month
Sales by region	→ SUM(sales) grouped by region
Customer segmentation	→ grouping customers by behavior
Year-over-year growth	→ comparing this year vs last year
Product performance	→ ranking products by revenue/units

Simple OLAP Architecture

```
[OLTP Sources]
  |
  ▼
ETL / ELT (extract, transform, load)
  |
  ▼
[Data Warehouse] — star schema: FACT + DIMENSION tables
  |
  ▼
Aggregated analytical queries
  |
  ▼
[Power BI / Tableau / Reports]
```

Section 6 — OLTP vs OLAP

Side-by-side comparison + memorable analogy

Feature	OLTP	OLAP
Full form	Online Transaction Processing	Online Analytical Processing
Purpose	Run day-to-day operations	Analyze data for decisions
Users	Front-line staff, applications	Analysts, managers, executives
Data	Current, up-to-date	Historical, aggregated
Operations	INSERT / UPDATE / DELETE (many, small)	SELECT with aggregation (few, large)
Query type	Simple, short	Complex, long-running
Transactions	Short, ACID-compliant	Not transactional; read-mostly
Data volume (per query)	Small (a few rows)	Large (millions of rows scanned)
Schema	Highly normalized (3NF)	Denormalized (star/snowflake)
Normalization	High	Low (intentionally denormalized)
Historical data	Little to none	Extensive
Example	ATM withdrawal, order placement	Monthly revenue dashboard

OLTP

Running the business.
Recording what is happening right now.

OLAP

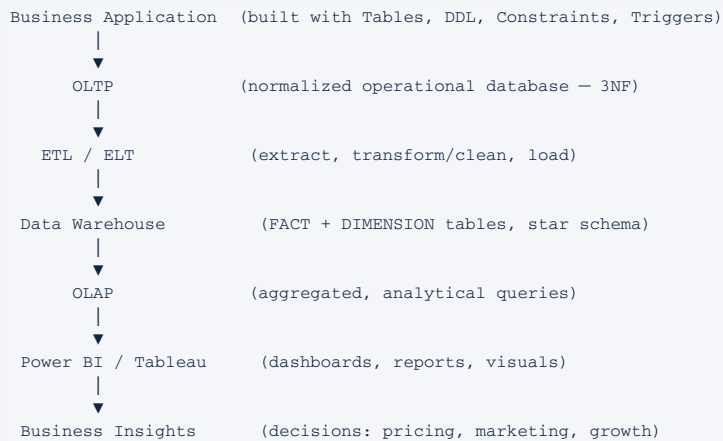
Analyzing the business.
Understanding what has happened over time.

IN VERY SIMPLE LANGUAGE

OLTP is the **cash register** — fast, one sale at a time, always up to date. OLAP is the **quarterly business report** — slower, looks back over thousands of sales, and tells you the story behind the numbers.

Section 7 — Complete Data Analytics Architecture

How every concept in this handbook connects end to end



Concept	Where it fits
Tables & DDL	Define the structure of both the OLTP app database and the warehouse tables.
Relationships (PK/FK, 1:1, 1:N, M:N)	Keep OLTP data consistent; also link FACT tables to DIMENSION tables in the warehouse.
Triggers	Run inside the OLTP database to audit/validate data as it is created.
ETL/ELT	Moves and reshapes OLTP data into the warehouse's denormalized structure.
OLTP	The source of truth for live operational data.
Data Warehouse	Central, historical, query-optimized store built for analysis.
OLAP	The querying/aggregation layer over the warehouse.
BI tools	Turn OLAP query results into dashboards for humans to act on.

Section 8 — Normalization & Denormalization

1NF • 2NF • 3NF • Why OLAP denormalizes

Normalization is the process of organizing tables to reduce data redundancy and improve data integrity.

Normal Form	Rule
1NF	Every column holds atomic (single, indivisible) values — no repeating groups or lists in one cell.
2NF	Must be in 1NF, and every non-key column depends on the <i>whole</i> primary key (relevant when the PK is composite).
3NF	Must be in 2NF, and no non-key column depends on another non-key column (no "transitive" dependency).

Customer / Order example

Unnormalized: one orders table storing `customer_name` and `customer_address` repeated on every single order row.

Normalized (3NF): a separate `customer` table holding the name/address once, and an `orders` table storing only `customer_id` as a foreign key — no repeated/redundant text.

WHY OLTP IS NORMALIZED

Normalization avoids storing the same fact twice, so an UPDATE only ever needs to touch one row (e.g. update a customer's address once, not on every order). This keeps writes fast, consistent, and free of anomalies — exactly what a transactional system needs.

Denormalization

Denormalization intentionally re-introduces redundancy — for example, storing the customer's name directly inside a sales fact row, rather than requiring a join every time. It trades some redundancy/write complexity for much faster read performance.

WHY ANALYTICAL SYSTEMS DENORMALIZE

OLAP queries mostly **read** huge volumes of data and rarely update individual rows. Fewer joins means faster aggregation across millions of rows — exactly what dashboards and reports need. Since OLAP data is loaded in batches (via ETL) rather than transaction-by-transaction, the "risk" of redundancy is easier to manage than in OLTP.

Section 9 — Fact and Dimension Tables

The building blocks of dimensional modeling / star schema

Fact Table

Stores the **measurable, numeric events** of the business — the "what happened, how much."

- Sales amount
- Quantity
- Discount

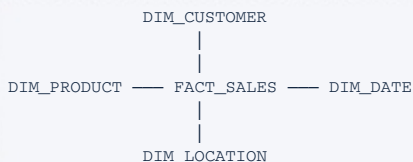
A fact table is usually very long (millions of rows) but narrow, mostly containing numbers and foreign keys pointing to dimension tables.

Dimension Table

Stores the **descriptive context** around a fact — the "who, what, when, where."

- Customer (name, segment, city)
- Product (name, category, brand)
- Date (day, month, quarter, year)
- Location (store, region, country)

Star Schema



HOW THIS RELATES TO OLAP / DATA WAREHOUSES

Every OLAP query is essentially: "aggregate a measure from the FACT table, sliced by one or more DIMENSIONS." For example:

`SUM(sales_amount) from FACT_SALES, GROUP BY DIM_DATE.month and DIM_LOCATION.region` → "monthly revenue by region." This star shape (one fact, several dimensions radiating out) is why it's called a **star schema** — it is the standard design pattern for a data warehouse and what BI tools like Power BI expect to query against.

Section 10 — Interview & Exam Questions

10 MCQs • 10 Conceptual • 10 SQL Coding — Answer Key at the end

Part A — Multiple Choice Questions (MCQs)

1. Which of the following is NOT a DDL command?
A) CREATE B) ALTER C) UPDATE D) TRUNCATE
2. Which statement permanently removes both the data and the table structure?
A) DELETE B) TRUNCATE C) DROP D) REMOVE
3. Which constraint ensures a column can never store duplicate values but still allows NULL?
A) PRIMARY KEY B) UNIQUE C) CHECK D) NOT NULL
4. In a One-to-Many relationship between CUSTOMER and ORDERS, where does the foreign key go?
A) CUSTOMER table B) ORDERS table C) A separate junction table D) Either table, it doesn't matter
5. Which trigger timing can modify the incoming value before it is saved to the table?
A) AFTER INSERT B) BEFORE INSERT C) AFTER UPDATE D) AFTER DELETE
6. In a DELETE trigger, which pseudo-row is available?
A) NEW only B) OLD only C) Both OLD and NEW D) Neither
7. Which of these is a characteristic of OLTP, not OLAP?
A) Historical data B) Denormalized schema C) High normalization D) Complex aggregations
8. A star schema is made up of:
A) Only fact tables B) Only dimension tables C) One fact table and several dimension tables D) Many fact tables and one dimension table
9. Which data type would you use for a product price?
A) FLOAT B) DECIMAL C) VARCHAR D) BOOLEAN
10. Can a trigger be invoked manually by a user via a CALL-like statement?
A) Yes, always B) No, never C) Only AFTER triggers D) Only in SQL Server

Part B — Conceptual Questions

1. Explain the difference between DDL and DML with one example each.
2. Compare DROP, TRUNCATE, and DELETE in terms of what they remove and whether they can be rolled back.
3. What is the difference between a primary key and a foreign key?
4. What is referential integrity, and how is it enforced?
5. Explain the difference between a trigger and a stored procedure.
6. Explain OLTP vs OLAP using a real-world analogy.
7. What is the difference between normalization and denormalization, and when would you use each?
8. What is a junction/bridge table and why is it needed for many-to-many relationships?
9. Explain the purpose of a fact table and a dimension table in a data warehouse.
10. What is the difference between BEFORE and AFTER triggers, with an example use case for each?

Part C — SQL Coding Questions

1. Write a CREATE TABLE statement for a `products` table with columns: `product_id` (primary key), `name` (mandatory text), `price` (must be greater than 0), and `stock_quantity` (default 0).
2. Write an ALTER TABLE statement to add a NOT NULL column `category` to the `products` table.
3. Write a CREATE TABLE statement for two tables, `department` and `employee`, modeling a One-to-Many relationship between them.
4. Write the SQL to create a junction table representing a Many-to-Many relationship between `students` and `courses`.
5. Write a CHECK constraint that ensures a column called `age` is always 18 or above.
6. Write a CREATE TRIGGER statement (MySQL syntax) that logs a row into `product_price_audit` whenever a product's price is updated.
7. Write an ALTER TABLE statement to rename the column `phone` to `mobile_no`.
8. Write a CREATE VIEW statement that shows only employees earning more than 50,000.
9. Write the SQL to add a UNIQUE constraint on the `email` column of an existing `users` table.
10. Write a query using GROUP BY that would be typical of an OLAP report: total sales amount per region from a `FACT_SALES` table joined to `DIM_LOCATION`.

Answer Key & Explanations

Part A — MCQ Answers

#	Answer	Explanation
1	C) UPDATE	UPDATE modifies data, so it's DML, not DDL.
2	C) DROP	DROP removes the table structure entirely; TRUNCATE keeps the structure.
3	B) UNIQUE	PRIMARY KEY disallows NULL; UNIQUE typically allows one NULL.
4	B) ORDERS table	The FK sits on the "many" side because each order has exactly one customer.
5	B) BEFORE INSERT	BEFORE triggers can modify NEW values prior to the insert being saved.
6	B) OLD only	The row is gone after DELETE, so only OLD (the pre-delete value) exists.
7	C) High normalization	OLTP is normalized for fast, consistent writes; OLAP is typically denormalized.
8	C) One fact table and several dimension tables	This is the definition of a star schema.
9	B) DECIMAL	DECIMAL is exact — required for money; FLOAT can introduce rounding errors.
10	B) No, never	Triggers only fire automatically in response to their defined event.

Part B — Conceptual Answers (summary)

- DDL defines structure (`CREATE TABLE`); DML changes data (`INSERT INTO`).
- DROP removes table + data (DDL, not rollback-able); TRUNCATE removes all rows, keeps structure (DDL, minimal logging); DELETE removes matched rows, keeps structure, and can be rolled back (DML).
- A primary key uniquely identifies rows in its own table; a foreign key references a primary/unique key in another table to link the two.
- Referential integrity means a foreign key value must match an existing key in the referenced table; enforced automatically by the FOREIGN KEY constraint.
- A stored procedure is called explicitly by the user/app; a trigger fires automatically on a data event and cannot be called directly.
- OLTP = running the business (cash register, real-time); OLAP = analyzing the business (quarterly report, historical).
- Normalization reduces redundancy for consistent, fast writes (used in OLTP); denormalization adds controlled redundancy for faster reads (used in OLAP/warehouses).
- A junction table breaks an M:N relationship into two 1:N relationships and can store attributes about the relationship itself (e.g. enrollment date).
- A fact table stores numeric, measurable events (sales amount); a dimension table stores descriptive context (customer, product, date, location) used to slice/filter the facts.
- BEFORE triggers run before the change is applied (good for validation); AFTER triggers run after the change is committed (good for auditing/logging).

Part C — SQL Coding Answers

```

-- 1.
CREATE TABLE products (
    product_id    INT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    price         DECIMAL(10,2) CHECK (price > 0),
    stock_quantity INT DEFAULT 0
);

-- 2.
ALTER TABLE products ADD category VARCHAR(50) NOT NULL DEFAULT 'GENERAL';

-- 3.
CREATE TABLE department (
    dept_id    INT PRIMARY KEY,
    dept_name  VARCHAR(50)
);
CREATE TABLE employee (
    emp_id    INT PRIMARY KEY,
    name      VARCHAR(50),
    dept_id   INT,
    FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);

-- 4.
CREATE TABLE student_course (
    student_id INT,
    course_id  INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

-- 5.
ALTER TABLE users ADD CONSTRAINT chk_age CHECK (age >= 18);

-- 6.
DELIMITER $$
CREATE TRIGGER trg_price_audit
AFTER UPDATE ON products
FOR EACH ROW
BEGIN
    IF OLD.price <> NEW.price THEN
        INSERT INTO product_price_audit (product_id, old_price, new_price, changed_at)
        VALUES (OLD.product_id, OLD.price, NEW.price, NOW());
    END IF;
END$$
DELIMITER ;

-- 7.
ALTER TABLE employee RENAME COLUMN phone TO mobile_no;

-- 8.
CREATE VIEW high_earning_employees AS
SELECT * FROM employee WHERE salary > 50000;

-- 9.
ALTER TABLE users ADD CONSTRAINT uq_email UNIQUE (email);

-- 10.
SELECT l.region, SUM(f.sales_amount) AS total_sales
FROM FACT_SALES f
JOIN DIM_LOCATION l ON f.location_id = l.location_id
GROUP BY l.region;

```

Section 11 — Quick Revision Sheet

One-glance cheat sheet for the day before your exam/interview

DDL

CREATE
ALTER
DROP
TRUNCATE

DML

INSERT
UPDATE
DELETE

DQL

SELECT

DCL

GRANT
REVOKE

TCL

COMMIT
ROLLBACK

TRIGGERS

BEFORE / AFTER
INSERT / UPDATE / DELETE
OLD / NEW

RELATIONSHIPS

1:1
1:N
M:N

OLTP

Transactions
Current data
Normalized
Operational

OLAP

Analysis
Historical data
Aggregations
Data Warehouse

SAY IT IN ONE LINE (PRACTICE OUT LOUD)

- **DDL** defines structure. **DML** changes data. **DQL** reads data.
- **PRIMARY KEY** identifies a row; **FOREIGN KEY** links to another table's key.
- **TRUNCATE** empties a table fast; **DROP** deletes the table itself; **DELETE** removes chosen rows and can be rolled back.
- A **TRIGGER** fires automatically on INSERT/UPDATE/DELETE — it is never called manually.
- **OLTP** runs the business (fast, current, normalized). **OLAP** analyzes the business (slow, historical, denormalized).
- A **FACT** table holds numbers; a **DIMENSION** table holds descriptive context.

Final Revision Checklist

Tick each box only once you can explain it out loud without notes

- ☐ DDL commands (CREATE, ALTER, DROP, TRUNCATE)
- ☐ Constraints (PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, DEFAULT, CHECK)
- ☐ Primary & Foreign Keys
- ☐ Relationships (1:1, 1:N, M:N)
- ☐ DROP vs TRUNCATE vs DELETE
- ☐ Triggers (what they are & why they're used)
- ☐ OLD vs NEW
- ☐ BEFORE vs AFTER
- ☐ OLTP (definition, characteristics, examples)
- ☐ OLAP (definition, characteristics, examples)
- ☐ OLTP vs OLAP comparison
- ☐ Normalization (1NF, 2NF, 3NF) & Denormalization
- ☐ Fact & Dimension Tables / Star Schema
- ☐ Data Warehouse Architecture (OLTP → ETL → Warehouse → OLAP → BI)
- ☐ SQL Interview Questions (MCQ, conceptual, coding)

YOU'RE READY WHEN...

You can draw the full architecture diagram (Section 7) from memory, explain OLTP vs OLAP in one sentence each, and write a CREATE TABLE with at least three different constraints without looking anything up.